

Image Classification using CNNs

Loïc Crittin

Lana Tannira

Pau Verdejo

Elliot Eriksson

Jonathan Eriksson

Abstract

Convolutional Neural Networks (CNNs) are widely used for image classification by learning hierarchical visual features directly from image data. In this project, a CNN-based image classification pipeline in Python using PyTorch was implemented and evaluated. Starting from a baseline architecture, the effects of training parameters, weight initialization, normalization methods, activation functions, and data augmentation were investigated. During the experiment, multiple independently initialized networks were trained, and then their predictions were combined in an ensemble to improve robustness and evaluate performance variation. The models are evaluated on CIFAR10 classification accuracy. Qualitative analysis is done using linear probing of a model trained on CIFAR10 and a 64x64 subset of imagenet, that closely matches cifar10.

1. Introduction

This report presents the work of Group 6 in the Image Classification project for the course TSBB19, Machine Learning for Computer Vision. The project investigates how several common neural-network design and training choices affect the performance of GoalNet on image-classification tasks. In particular, we evaluate the effects of hyperparameters and weight initialization, normalization methods, independently trained model ensembles, linear probing, and data augmentation through conventional transformations and mixup.

Rather than aiming to fine-tune a model solely to obtain the highest possible classification accuracy, the purpose of this project is to examine how individual design choices influence training behaviour, learned feature representations, and test-set generalization. The experiments are therefore designed as controlled comparisons in which the remaining training settings are kept fixed whenever possible. This makes it possible to relate observed changes in performance to a specific modelling choice.

Section 2 describes the experimental procedures and training configurations. The datasets and evaluation procedure are introduced in Section 3, followed by the quanti-

tative and qualitative results in Section 4. Finally, Section 5 summarizes the main findings and their implications for the design of image-classification networks.

2. Methodology

2.1. GoalNet Training Hyperparameters

We evaluated GoalNet’s sensitivity to hyperparameters by systematically varying the learning rate (η), batch size (B), weight decay, and weight initialization from a fixed baseline ($\eta = 10^{-3}$, $B = 256$, zero weight decay, default PyTorch initialization). For weight initialization, we compared Kaiming normal and uniform [5], Xavier uniform [4], and a hybrid scheme combining Kaiming (for convolutional layers) with Xavier (for fully connected layers). Models were trained on CIFAR-10 [7] for 25 epochs. The configuration yielding the highest test accuracy was adopted as the reference for all subsequent experiments.

2.2. Ensemble of Independently Trained GoalNets

To assess run-to-run variance and the benefits of model ensembling, we trained three instances of GoalNet completely independently. Each network utilized the identical training configuration but differed in its random weight initialization and stochastic mini-batch sampling. The individual softmax probability outputs on the test set were then averaged to form a single ensemble prediction (Section 4). This procedure measures both the individual network performance variance (mean and standard deviation) and the accuracy gain obtained by fusing predictions.

2.3. Effect of Normalization

The impact of feature normalization was evaluated by inserting either Batch Normalization [6], Group Normalization (with 4 groups) [9], or Layer Normalization [2] immediately after each of the three convolutional layers, before max-pooling and activation. Layer Normalization was applied over the exact dimensions of each intermediate activation map ($32 \times 32 \times 32$, $32 \times 16 \times 16$, and $64 \times 8 \times 8$). To account for training stochasticity (Section 2.2), each scheme—alongside a non-normalized baseline—was evaluated using an ensemble of three independently trained net-

works. To isolate the normalization effect, all configurations were identically trained for 40 epochs using AdamW [8] ($\eta = 10^{-3}$, weight decay 10^{-4} , batch size 128) and ELU activations [3].

2.4. Linear Probing

Linear Probing [1] is a technique to better understand a deep neural network. The probe is inserted after an activation in the network, and a linear classifier is trained on the output from that activation. This makes it possible to see how each layer improves the accuracy of the network. In order to use linear probing, the entire network is trained, and then the linear probes are inserted at different points of the network. In this experiment we chose to probe the three first activation functions and train a linear classifier on their output. In section 4, the results are discussed, and a visualization of the outputs of the two first activations is provided in Appendix A.

2.5. Data Augmentation and Mixup

We investigated data augmentation using both conventional image transformations and mixup. The conventional augmentation consisted of random cropping with four pixels of padding and random horizontal flipping. These transformations were only applied to the training set.

Mixup is a data augmentation and regularization method proposed by Hongyi Zhang et al. [10]. Instead of training only on the original samples, virtual training examples are constructed by linearly interpolating two randomly selected samples and their corresponding targets.

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j \quad (1)$$

$$\tilde{y} = \lambda y_i + (1 - \lambda)y_j \quad (2)$$

where the mixing coefficient is sampled according to:

$$\lambda \sim \text{Beta}(\alpha, \alpha) \quad (3)$$

The hyperparameter α controls the strength of the interpolation. For each minibatch, the samples were randomly permuted and mixed with the corresponding shuffled samples. Since GoalNet uses LogSoftmax together with NLLLoss, the mixed-target loss was implemented as seen in equation 4.

$$\mathcal{L}_{mix} = \lambda \mathcal{L}(f(\tilde{x}), y_i) + (1 - \lambda) \mathcal{L}(f(\tilde{x}), y_j) \quad (4)$$

This formulation is equivalent to evaluating the loss using the linearly interpolated target $\tilde{y}$.

For the controlled augmentation experiments, all other training settings were kept fixed. The settings used were: Group normalization, the default weight initialization, a learning rate of 0.001, batch size of 256, weight decay of 5×10^{-4} , 20 training epochs and random seed being 42. We evaluated mixup for $\alpha \in \{0.2, 0.4, 1.0\}$.

3. Description of dataset and evaluation

3.1. CIFAR10

The CIFAR10 dataset consists of 60000 images, of which 50000 being training images and 10000 being test images. The images are divided into 10 classes with 6000 images in each class. The classes in the dataset are as listed: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. Each image has a spatial dimension of 32 x 32, and all images are colorized.

3.2. Imagenet

A subset of the imagenet dataset was used which is constructed to have the same 10 classes as CIFAR10 but with a spatial dimension of 64 x 64. Thereby allowing seamless implementation with the model built around using CIFAR10 and allows testing with different image sizing.

4. Experimental results

4.1. Evaluation

In order to evaluate the answers and the accuracy of the models, a CIFAR10 classification accuracy method was used where the models are fed the set of 10000 test images. By then comparing the predicted output from the model with the class assigned to each test image, it is possible to garner whether the answer is correct or incorrect. The correct answers are tallied up, and the accuracy is found by dividing the tally by the amount of images in the training set.

4.2. GoalNet learning parameters

The experiment allowed for the evaluation of the sensitivity of GoalNet to weight initialization, learning rate, weight decay and batch size. Unless otherwise specified, experiments were done using 25 training epochs.

Table 1. Effect of GoalNet learning parameters.

Setting	Value	Acc. (%)
Baseline	$\eta = 10^{-3}$, $B = 256$, $WD = 0$	72
LR	3×10^{-4}	72
LR	3×10^{-3}	69
Batch size	64	70
Batch size	128	72
Batch size	512	72
Weight decay	10^{-3}	72
Weight decay	10^{-2}	70

The learning rate $\eta = 0.003$ reduced test accuracy down to 69% from our baseline of 72%, suggesting that the updates were too large. In contrast, $\eta = 0.0003$ achieved the same accuracy as the reference configuration but converged more slowly. Batch sizes 128, 256, and 512 achieved comparable test accuracy, while batch size 64 performed

worse. Weight decay 10^{-3} preserved the reference accuracy, whereas stronger regularization (10^{-2}) reduced performance.

Table 2. Effect of weight initialization on GoalNet.

Initialization	Acc. (%)
Kaiming normal	73
Kaiming uniform	71
Xavier uniform	72
Kaiming U. (conv.) + Xavier U. (FC)	71

Kaiming normal initialization achieved the highest test accuracy at 73%. Xavier uniform was close to the reference performance, whereas Kaiming uniform and the hybrid Kaiming/Xavier initialization reached 71%. Thus, the initialization method had a modest but measurable effect on GoalNet generalization, and Kaiming normal was selected for this architecture.

4.3. Ensemble of independently trained GoalNets

Three GoalNets were initialized and trained independently. During inference, their softmax probability outputs were averaged:

$$p_{\text{ens}}(y | x) = \frac{1}{3} \sum_{m=1}^3 p_m(y | x). \quad (5)$$

Table 3. Individual and ensemble test accuracies.

Model	Test acc. (%)
GoalNet 1	80.42
GoalNet 2	80.71
GoalNet 3	80.13
Mean individual accuracy	80.42
Standard deviation	0.24
Ensemble accuracy	84.87

The individual models achieved similar accuracies, with a standard deviation of 0.24 percentage points. However, their errors were not identical. Averaging their softmax outputs improved test accuracy to 84.87%, which is 4.45 percentage points above the mean individual accuracy and 4.16 percentage points above the best individual model.

4.4. Effect of Normalization

Table 4 reports the effect of adding a normalization layer at every convolutional stage of GoalNet, evaluated using the ensemble protocol described in Section 2.3 (3 independently trained networks per configuration, AdamW, $\eta = 10^{-3}$, weight decay 10^{-4} , batch size 128, ELU activation, 40 epochs).

At the level of individual networks, normalization improved test accuracy over the unnormalized baseline: BatchNorm achieved the highest mean individual accuracy

Table 4. Effect of normalization on GoalNet, evaluated using an ensemble of 3 independently trained networks per configuration.

Normalization	Net 1	Net 2	Net 3	Mean $\pm$ Std (%)	Ensemble (%)
None	80.42	80.71	80.13	80.42 ± 0.24	84.87
BatchNorm	81.80	81.20	81.15	81.38 ± 0.30	84.53
GroupNorm	80.29	81.05	81.48	80.94 ± 0.49	84.56
LayerNorm	80.09	80.70	80.68	80.49 ± 0.28	84.06

(81.38%), followed by GroupNorm (80.94%) and LayerNorm (80.49%), compared with 80.42% without normalization. This is consistent with the established role of normalization in stabilizing intermediate activation statistics during training.

At the ensemble level, however, this ordering is reversed: the unnormalized configuration achieves the highest fused accuracy (84.87%), ahead of GroupNorm (84.56%), BatchNorm (84.53%) and LayerNorm (84.06%), despite starting from the lowest mean individual accuracy of the four. This can be explained by the larger accuracy gain obtained from fusing the unnormalized networks (+4.45 percentage points over their mean individual accuracy) compared with the normalized configurations (+2.62 to +3.65 percentage points), suggesting that the three unnormalized networks converge to more diverse solutions with less correlated errors, which is precisely the property that makes ensembling effective.

4.5. Linear Probing

In table 5 the results of linear probing are reported. For both datasets the network was trained for 50 epochs with a batch size of 64, all networks were trained without any data augmentation strategy. The accuracy of the predictions increase with every layer. Furthermore, the output after the

Table 5. Test accuracy after linear probing at different network depths.

Probe location	ImageNet64	CIFAR-10
Full network	73.71%	73.87%
First activation	42.63%	43.96%
Second activation	51.07%	50.75%
Third activation	62.29%	60.71%

first two activations was visualized to qualitatively analyze the layer output. The activations are shown in Appendix A. It is clear that some channels show high activation around the predicted object, interestingly enough it possible can see that for the frog and the cat the channels don’t correspond exactly, for example in channel six in image A there seems to be some activation around the face of the cat, but for the frog in image A the activation is focused around the background. The visualization also provides us some insight to what the network is ”looking at”. It is also possible to see that the visualization after the first activation function for

the frog looks more noisy, as in a lot of the channel outputs are random spots of activation, whereas for the cat all the channel outputs are very clear, which could imply that the cat is easier to classify.

4.6. Data Augmentation and Mixup

Table 6 presents the results of the controlled augmentation experiments. The model trained without augmentation obtained a test accuracy of 73.94%. Conventional augmentation using random cropping and horizontal flipping increased the accuracy to 77.07%.

All evaluated mixup configurations also improved the test accuracy compared with the non-augmentation model. The best mixup result was obtained using $\alpha = 1.0$, reaching 77.05% test accuracy, an improvement of 3.11 percentage points over the non-augmented baseline. The results for $\alpha = 0.2$ and $\alpha = 0.4$ were 76.38% and 76.48% respectively.

Table 6. Test accuracy for the augmentation experiments. All other training parameters were kept fixed.

Method	Test acc. (%)	Change (pp)
No augmentation	73.94	–
Basic augmentation	77.07	+3.13
Mixup ($\alpha = 0.2$)	76.38	+2.44
Mixup ($\alpha = 0.4$)	76.48	+2.54
Mixup ($\alpha = 1.0$)	77.05	+3.11

Increasing α resulted in a clear decrease in the weighted training accuracy from 77.76% for $\alpha = 0.2$ to 61.42% for $\alpha = 1.0$. At the same time, the test accuracy increased from 76.38% to 77.05%. This indicates that stronger interpolation made the training problem more difficult while improving generalization.

This behavior is consistent with the regularizing effect of mixup. The network is trained not only on the original training samples but also on virtual samples located between them, which encourages smoother predictions between training examples.

Conventional augmentation and mixup with $\alpha = 1.0$ achieved almost identical test accuracies, 77.07% and 77.05% respectively. The difference of 0.02 percentage points is too small to establish a clear advantage for either method from these experiments. However, both methods substantially outperformed the model trained without augmentation.

Only one random seed was evaluated for each configuration. Therefore, small differences between the augmentation methods should be interpreted cautiously.

5. Conclusion

In this project several design and training choices for GoalNet were evaluated on image classification tasks. The

experiments showed that learning rate, initialization, normalization and data augmentation can affect classification performance, although several configurations produced relatively similar results.

Ensembling independently trained GoalNet provided one of the clearest improvements, demonstrating the benefit of combining models with different prediction errors. The normalization experiments showed that BatchNorm achieved the highest mean accuracy for individual networks, while the unnormalized ensemble achieved the highest fused accuracy.

Linear probing showed increasing classification accuracy at deeper network layers, indicating increasingly discriminative feature representations. The activation visualizations also showed that different feature channels responded to different regions and structures in the input image.

Finally, both conventional data augmentation and mixup improved generalization compared with training without augmentation. The mixup experiments further showed that the strength of interpolation influenced the balance between training performance and test generalization.

A. Activation function visualizations



Figure 1. Visualization after the first activation function of all channels.

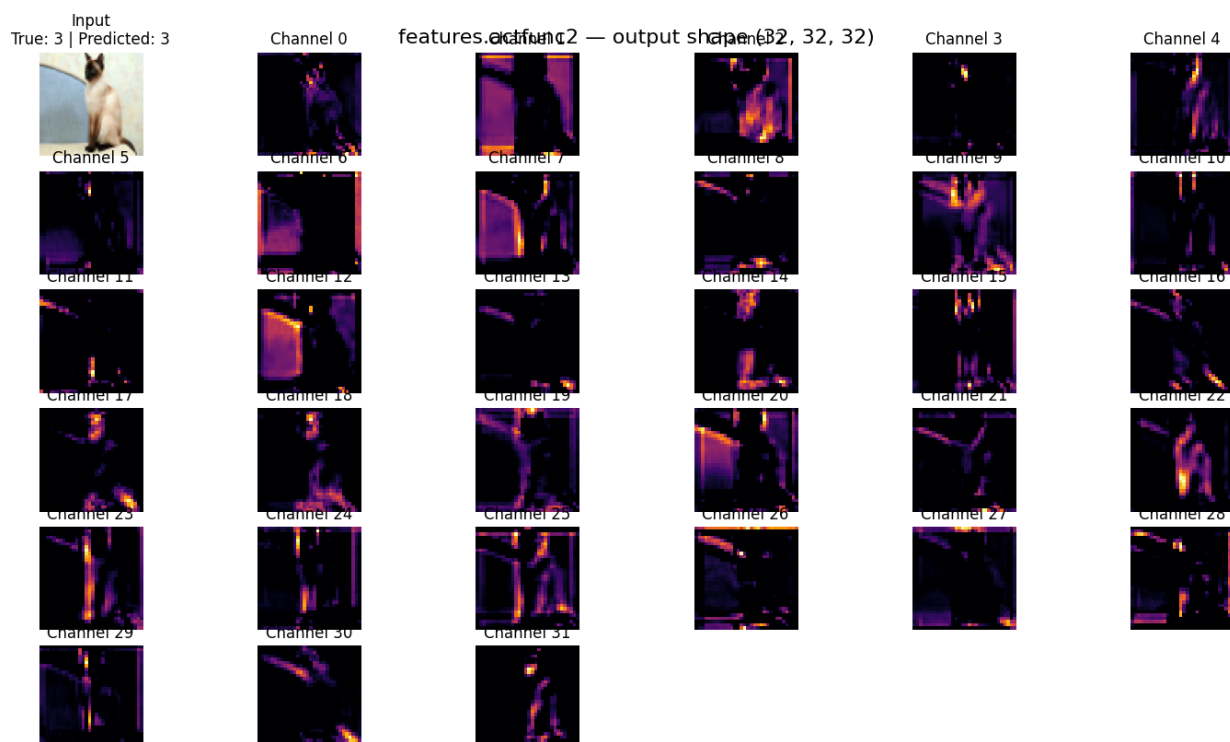


Figure 2. Visualization after the second activation function of all channels.

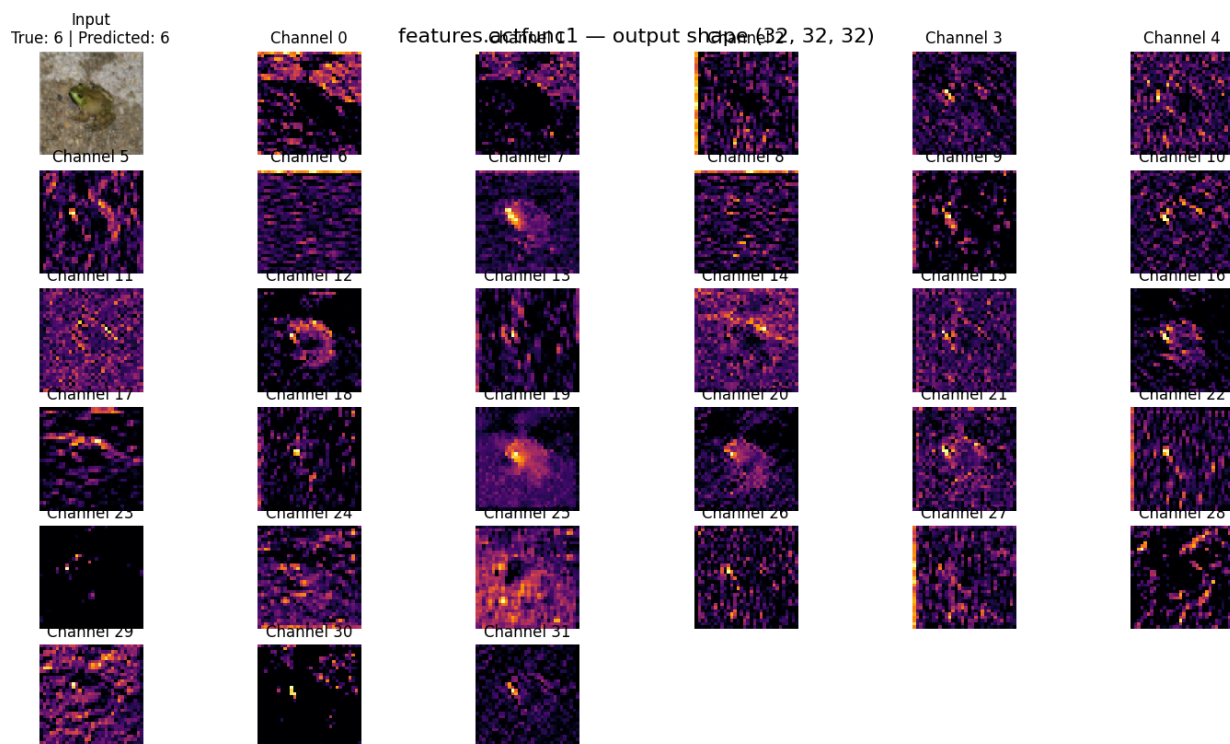


Figure 3. Visualization after the first activation function of all channels.

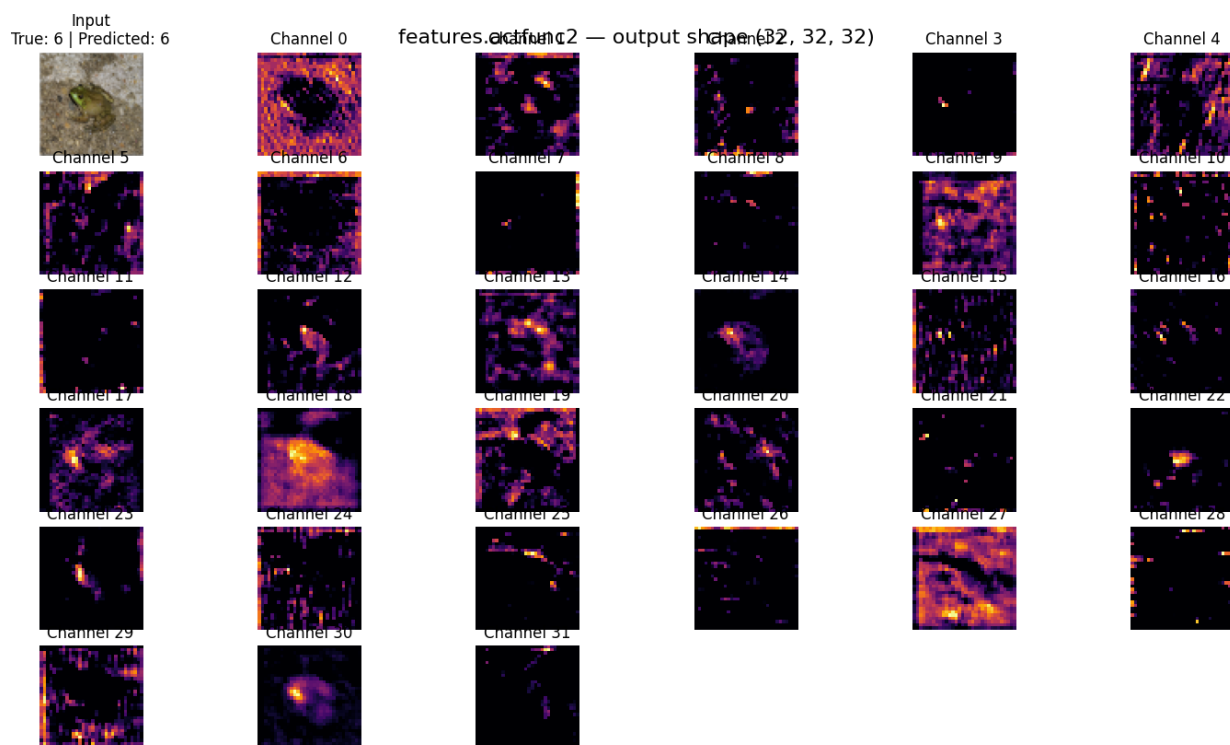


Figure 4. Visualization after the second activation function of all channels.

References

- [1] G. Alain. Understanding intermediate layers using linear classifier probes. 10 2016.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [3] D.-A. Clevert, T. Unterthiner, and S. Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). In *International Conference on Learning Representations*, 2016.
- [4] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [5] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- [6] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.
- [7] A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [8] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [9] Y. Wu and K. He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.
- [10] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018.